

Question 1

Objective

To report progress on the Pipeline 5 zoning-rule extraction system and its accompanying verification layer, which together convert municipal bylaw PDFs into GIS-safe, auditable zoning rules, and to confirm that the approach aligns with your requirements.

Attempts

We have developed and evaluated an end-to-end extraction pipeline on the Burnaby R1 (Small-Scale Multi-Unit Housing District) bylaw, comprising two components:



Pipeline 5 (extraction): Each bylaw page is partitioned into clause-style text blocks and regulatory-table image crops. A vision-language model then extracts atomic zoning rules — rule object, operator, value, unit, scope, and condition — with one cited source excerpt per rule. The rules are merged and deduplicated into a single rule registry.

Verification layer (trust gate): A deterministic checker re-reads each candidate rule against its cited evidence and confirms that the value, unit, operator, rule family, and scope are genuinely supported. Each rule is routed to one of three states — verified, needs-review, or rejected — and only verified rules enter the GIS-safe contract. Every decision is accompanied by a full audit trail.

On Burnaby R1, Pipeline 5 produced 142 candidate rules, which the verification layer subsequently evaluated (initial routing: 5 verified, 65 held for review, 72 rejected).

Evidence

Pipeline 5 extraction code and Burnaby rule registry: [code/prototype_pipeline_5/](#)

Verification layer and Burnaby results: [code/burnaby_rule_verification_prototype/](#)

Diagnosis

The extracted values are faithful to the source: the verifier measured an evidence-match rate of 1.00 and a value-grounding rate of 0.97 on Burnaby R1, indicating that the model is not fabricating numeric values.

The majority of non-verified rules are not extraction errors. The verification layer is intentionally conservative, holding text-block rules and table cells not yet covered by the GIS contract for human review rather than auto-accepting them.

The most significant correctable issue was a unit-encoding defect: square-metre values were emitted as "m²" while the contract expected "m2", which silently rejected every lot-area rule. This has been resolved through canonical unit normalization and a guard that flags any non-standard unit.

We have introduced generalized, municipality-agnostic extraction-quality guards — addressing bylaw cross-reference numbers incorrectly placed in the value field, rule-object/unit family mismatches, and reversed minimum/maximum operators — enforced at both the prompt and code levels. These ensure such issues are surfaced for review on any future bylaw rather than entering the registry unnoticed.

A remaining limitation is rule-object naming drift (for example, "rear_setback," "side_yard_width," and "separation_distance" referring to the same underlying families), which complicates downstream GIS consumption.

Proposed Next Steps

Canonicalize rule objects by mapping extraction-time names to a fixed GIS vocabulary while preserving the original text, ensuring uniform rule families for downstream use.

Relax the verification layer's table-scope checks, which currently depend on exact wording and over-route valid table rules to review.

Establish a cross-municipality regression benchmark (Burnaby and Vancouver) that validates canonical fields, to detect drift as the pipeline is applied to new bylaws.

Decision Ask

Do you agree with this overall approach, and are there any additional rules, fields, or features you would like the output to include?

Question 2

Objective

To determine how the verification layer can promote more source-supported rules from review_needed into verified while preserving the zero-false-verified safety target.

Attempts

We benchmarked the latest verifier against the Burnaby R1 gold rules, proposal cases, and adversarial cases. The current run reaches `candidate_recall = 1.00` and `verified_or_review_recall = 1.00`, but `verified_gold_recall` remains 0.41, with 9 verified rules and 83 rules still routed to review.

The existing verifier already checks value, unit, operator, rule family, scope, applies_to, condition, and exception evidence. It also keeps table proof traces and blocks poisoned adversarial cases, so the remaining challenge is not coverage but safe promotion.

Evidence

Benchmark report: `Burnaby_prototype/outputs/burnaby_r1_slim_pipeline5_registry/benchmark_report.md`

Review queue: `Burnaby_prototype/outputs/burnaby_r1_slim_pipeline5_registry/review_needed.json`

Verification code: Burnaby_prototype/src/burnaby_prototype/verification.py and rule_claims.py

Diagnosis

Biggest issue #1: text-rule verification is still too conservative. Many correct-looking text-block candidates are held for review because the verifier does not yet build strong span-level proof for scope, applies_to, condition, and operator wording in prose. This protects precision, but it limits automation.

Proposed Next Steps

Add a text-proof layer that extracts explicit evidence spans for value, unit, operator, rule object, scope, applies_to, and condition. An LLM or transformer can propose the spans, but deterministic checks must still validate the final support before any rule becomes verified.

Expand adversarial tests for wrong-scope values, min/max flips, nearby unrelated numbers, exception clauses, and unsupported applies_to claims before increasing verified recall.

Decision Ask

Are you comfortable with an AI-assisted proof-proposal step if the final verified decision remains deterministic, source-span-based, and benchmark-gated?

Question 3

Objective

To assess whether the verification layer can generalize beyond Burnaby R1 to additional BC bylaws without becoming overfit to one municipality, table style, or extraction vocabulary.

Attempts

We centralized shared vocabulary in a domain schema, added adapter normalization for Pipeline 5 handoff fields, separated not_used cross-reference artifacts from true rejections, and added regression tests for rule-object/unit mismatches and wrong-scope evidence.

We also fixed Pipeline 5 mismatch cases by using high-signal fields such as rule_key, subject, unit, table title, row header, and cell value before broad evidence text. This improved candidate recall without relaxing the verification gate.

Evidence

Normalization code: Burnaby_prototype/src/burnaby_prototype/zihao_adapter.py and domain_schema.py

Decision policy: Burnaby_prototype/src/burnaby_prototype/decision_policy.py

Regression tests: Burnaby_prototype/tests/test_slim_verifier.py

Diagnosis

Biggest issue #2: cross-bylaw normalization and schema drift remain the largest generalization risk. Different BC bylaws may use different names for the same rule family, encode units differently, place constraints in table headers instead of cells, or express exceptions in prose. If these mappings stay too local, the system may either reject too much or verify the wrong scope.

Proposed Next Steps

Create a municipality-agnostic rule ontology with per-bylaw configuration files, add a normalization_trace field explaining every canonicalization decision, and run the same benchmark suite across Burnaby plus at least two additional BC bylaws.

Build a small Streamlit dashboard to compare verified, review, rejected, and not_used outputs across bylaws, with filters for support gaps and proof traces so failures can be diagnosed without manually opening JSON files.

Decision Ask

Which BC bylaws or municipalities should be used next for the cross-bylaw validation benchmark, and should generalization testing be prioritized before expanding auto-verification?